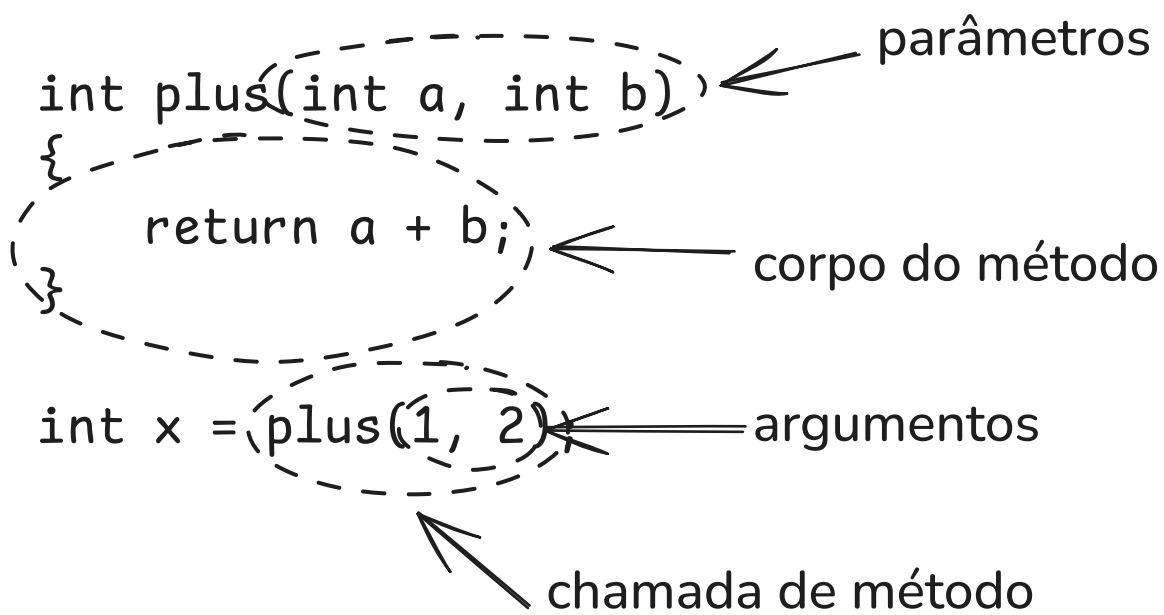


| Parâmetros

João Eduardo Montandon

DCC024

| Terminologia



| Correspondência

Como uma linguagem decide quais parâmetros recebem quais argumentos? 🤔

| Posicional

Casamento é feito conforme a ordem com que os argumentos são fornecidos.

```
int dividir(int num, int den) {  
    return num / den;  
}  
cout << dividir(4,2) << endl;
```

Algumas linguagens interpolam os parâmetros para facilitar o mapeamento.

```
- (int)dividir:(int)num por:(int)den {  
    return a / b;  
}  
  
[self dividir:4 por:2];
```

| Nominal

Argumentos são atrelados aos nomes dos parâmetros.

```
def dividir(num, den):  
    return num / den  
  
print(dividir(den=2, num=4))
```

| Valores-padrão

- Algumas linguagens permitem declarar valores padrão para seus parâmetros.
- O parâmetro recebe este valor caso nenhum argumento seja fornecido para ele.

```
def dividir(num, den=1):  
    return num / den  
print(dividir(4, 2))  
print(dividir(den=2, num=4))  
print(dividir(4))
```

Facilita em alguns casos, mas pode trazer problemas [\[1\]](#)

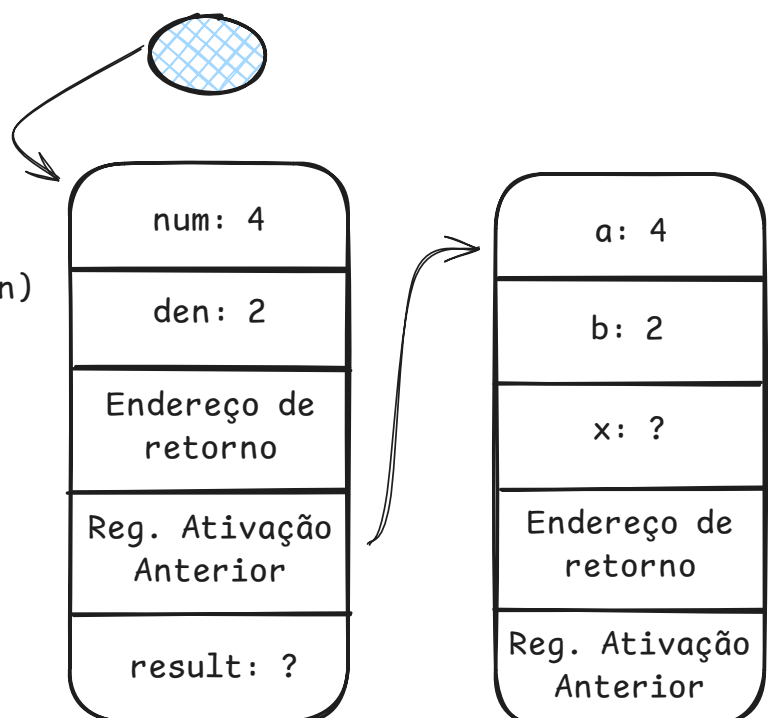
| Passagem De Parâmetros

| Por Valor

- Os parâmetros são variáveis locais no Registro de Ativação do método chamado.
- São inicializados com os mesmos valores dos argumentos.

```
int dividir(int num, int den)  
{  
    return num / den;  
}
```

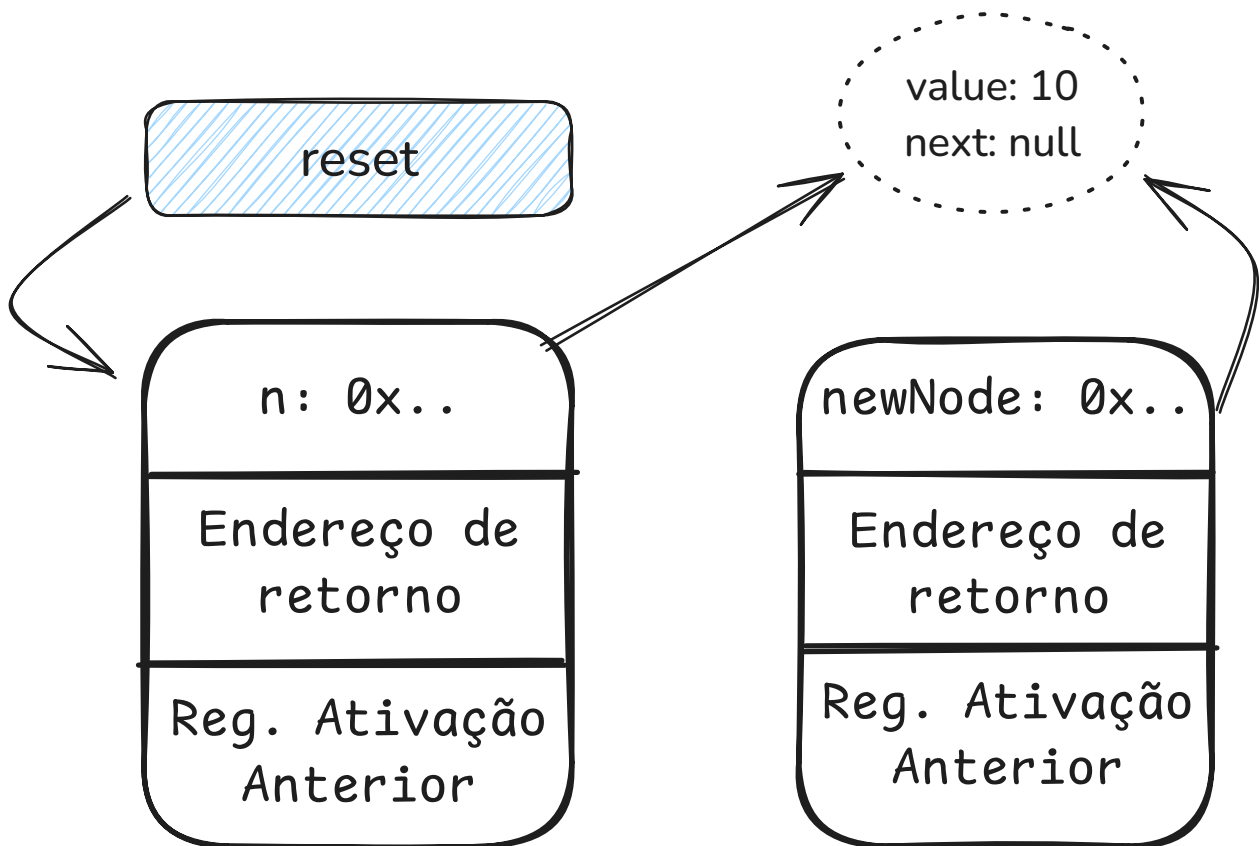
```
int a = 4;  
int b = 2;  
int x = dividir(a, b);
```



Em Java, **toda passagem é por valor**.

Mas e os objetos? 🤔

```
void reset(Node n) {  
    n.setValue(0);  
}  
void f() {  
    Node newNode = new Node(10);  
    reset(newNode);  
}
```



Como provar que a passagem é feita por valor? 🤔

```
public void resetNode(Node n) {
    n = null;
}

public void g() {
    Node test = new Node(10);
    System.out.println(test.value);
    resetNode(test);
    System.out.println(test.value);
}
```

Qual seria o equivalente para C/C++? 🤔

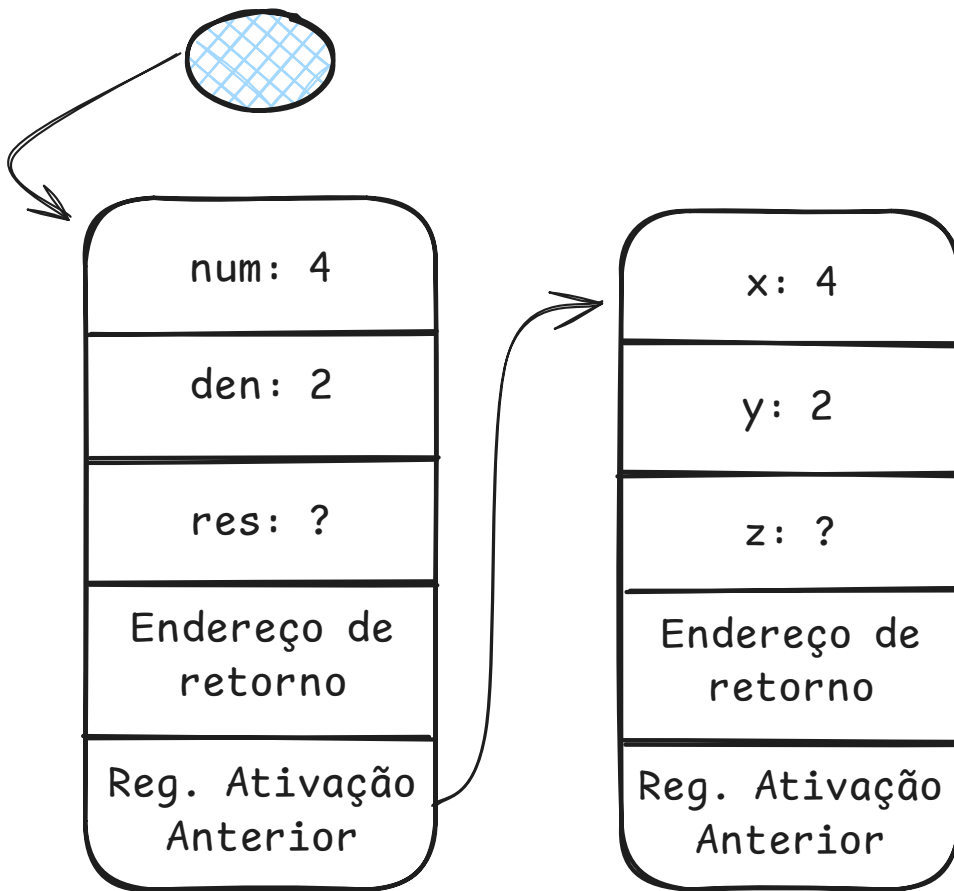
| Por Resultado

- O parâmetro funciona como uma variável local no RA do método chamado.
- Porém seu valor não é inicializado.
- Valor final do parâmetro é retornado para o respectivo argumento ao final do método.

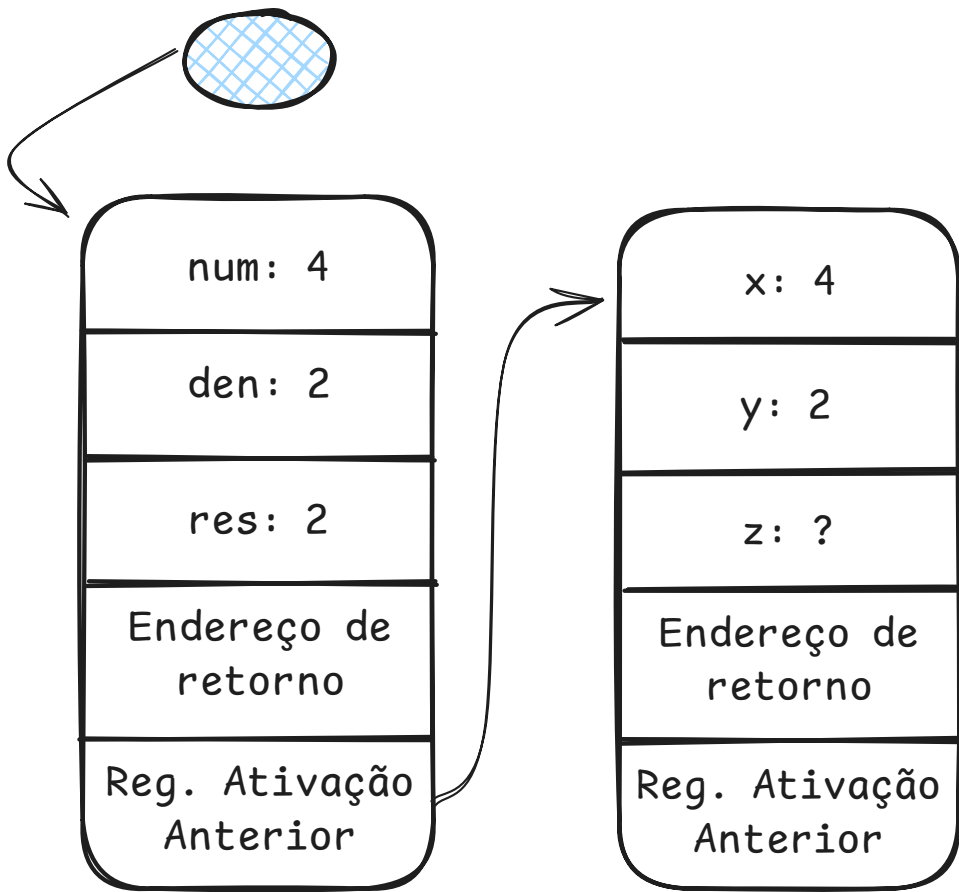
```
void dividir(int num, int den, by-result int res) {
    res = num / den;
}

void f() {
    int x = 4;
    int y = 2;
    int z;
    dividir(x, y, z);
}
```

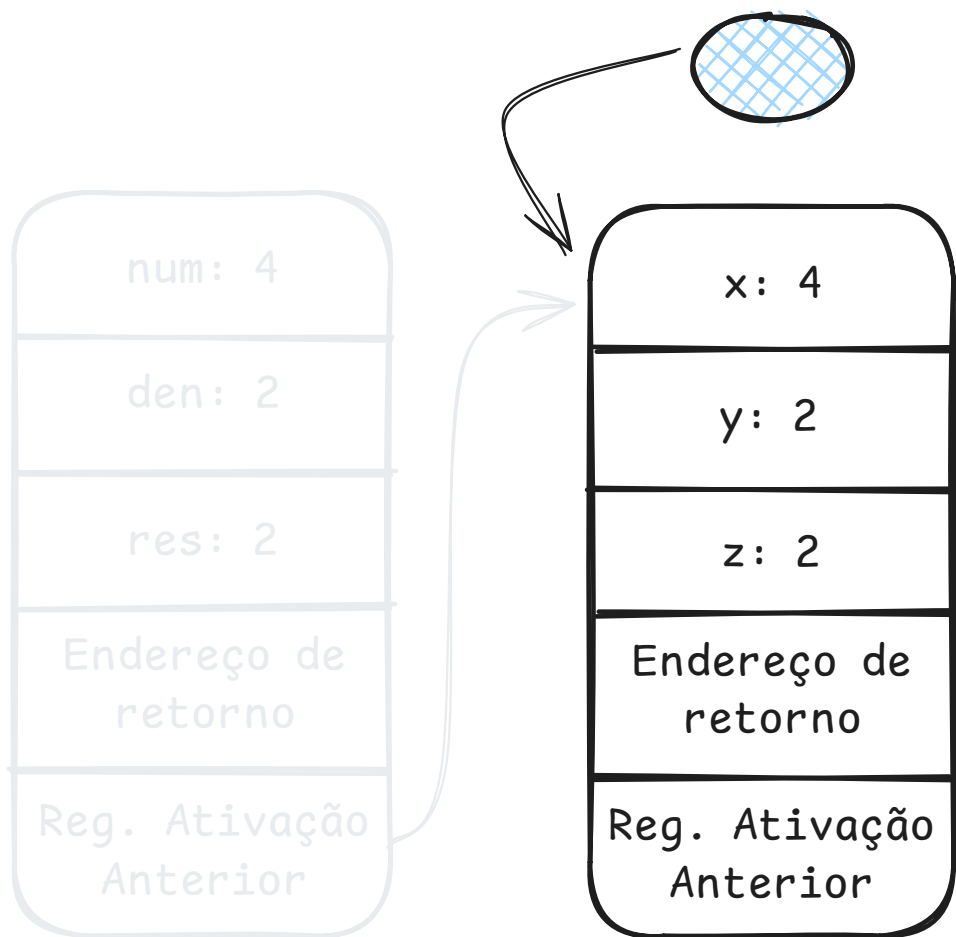
Início da execução de `dividir`.



Quando operação é realizada.



Fim da execução de `dividir`.



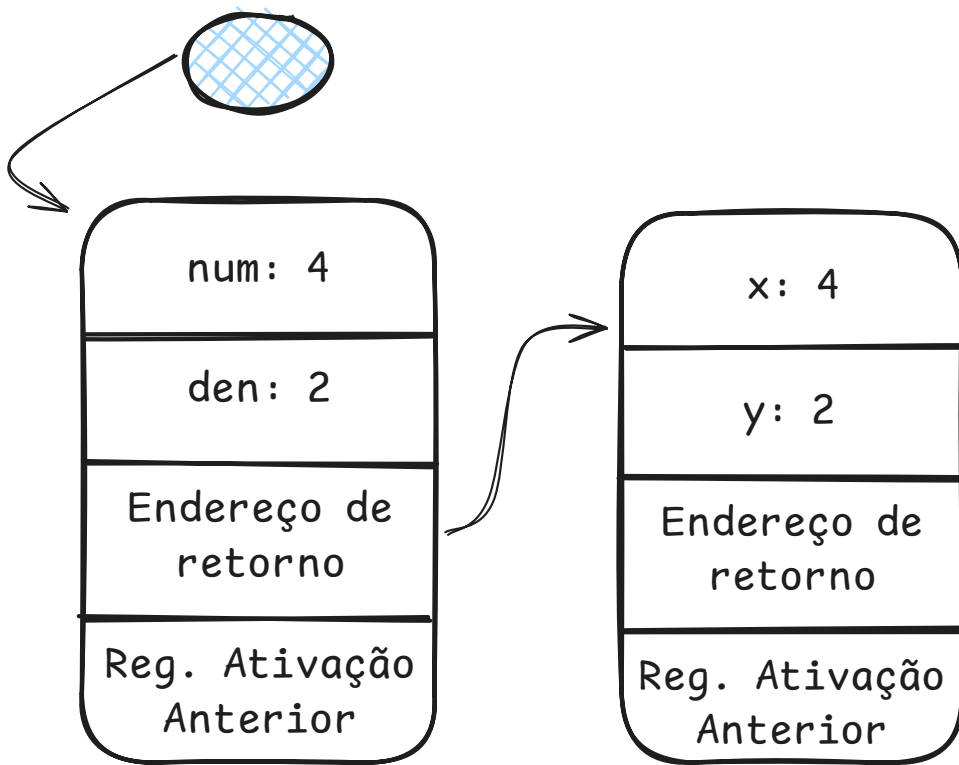
Por Valor-Resultado

- Por-Valor: o que acontece *antes do método* ser executado.
- Por-Resultado: o que acontece *após o método* ser executado.
- Por Valor-Resultado = Por-Valor + Por-Resultado.

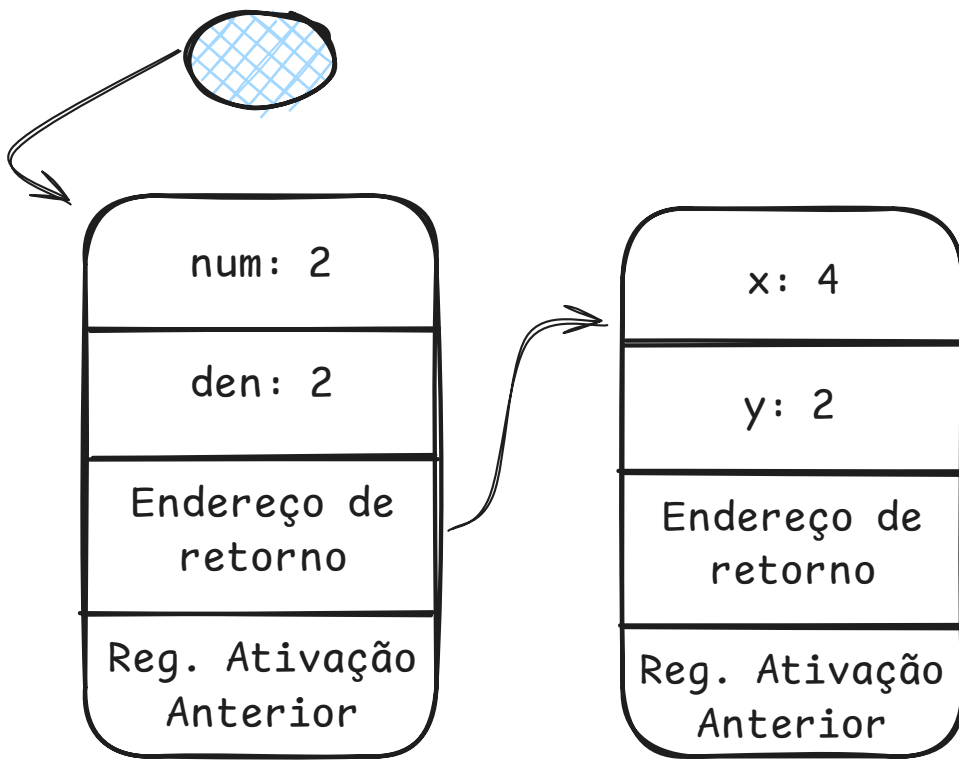
```
void dividir(by-value-result int num, int den) {  
    num /= den;  
}  
void f() {  
    int x = 4;  
    int y = 2;
```

```
    dividir(x, y);  
}
```

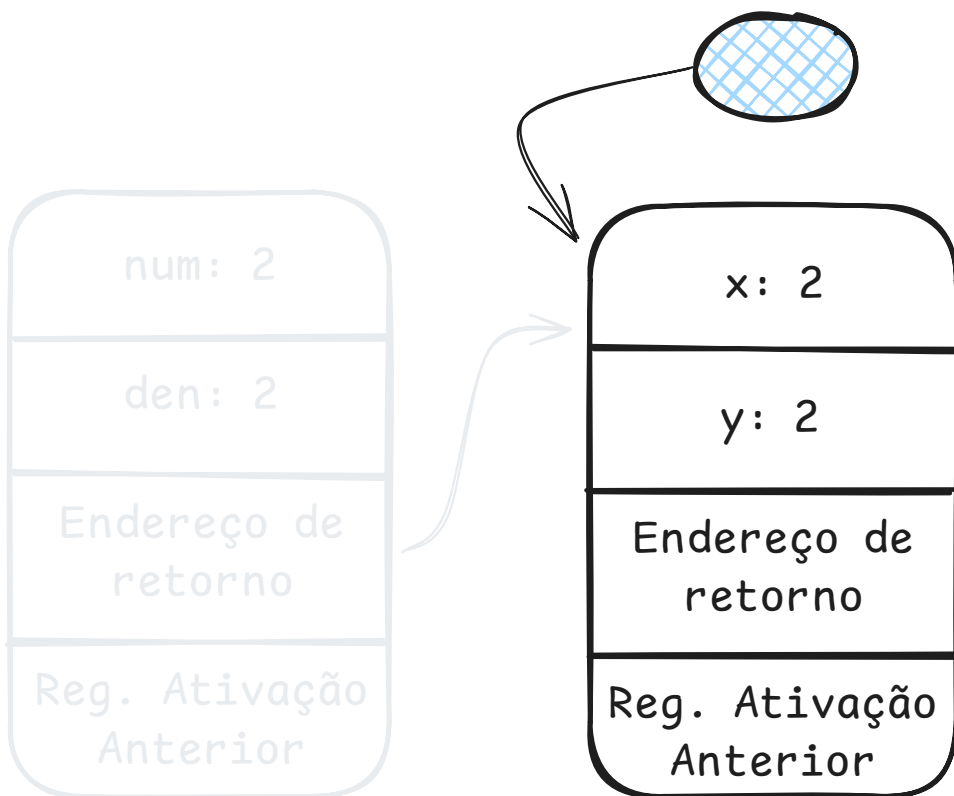
Início da execução.



Quando operação é realizada.



Fim da execução.

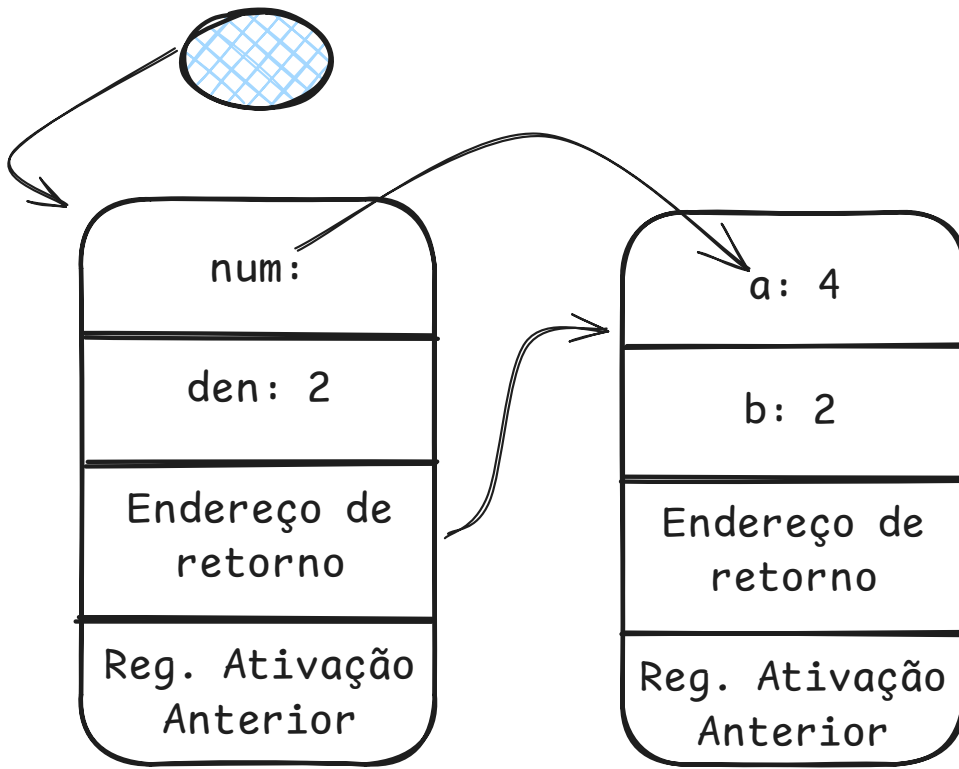


| Por Referência

O que é uma variável, realmente? 🤔

- Ocorre quando **lvalue** (referência) do argumento é atribuído ao parâmetro.
- Tanto parâmetro quanto argumento **estão no mesmo local**.

```
#include <stdio.h>
void dividir(int& num, int den) {
    num /= den;
}
int main() {
    int a = 4, b = 2;
    printf("a=%d, b=%d\n", a, b);
    dividir(a, b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```



É preciso tomar cuidado com os efeitos colaterais do uso da referência.

```
void refsum(int& n, int& ans) {  
    ans = 0;  
    int i = 1;  
    while(i ≤ n) ans += i++;  
}
```

```
int x = 10;  
refsum(x,x);
```

Qual o valor de `x`? 🤔

| Expansão De Macro

- Etapa **anterior** ao processo de compilação.
- Substitui a ocorrência do macro pelo seu corpo.

- Substitui também os parâmetros pelos argumentos.
- Na etapa de expansão de macro, **nenhum código é executado!**

```
#define MIN(X, Y) ((X) <= (Y) ? (X) : (Y))  
int x = 10;  
int y = 20;  
int a = MIN(x, y);
```



```
int a = ((x) < (y) ? (x) : (y));
```

Qual o valor de `a`?

```
#define MIN(X, Y) ((X) <= (Y) ? (X) : (Y))  
  
int x = 9;  
int y = 10;  
  
int a = MIN(++x, y);  
printf("%d %d %d\n", x, y, a);
```

O que vai ser impresso agora? 🤔

```
int a = MIN(++x, y++);
```



```
int a = ((++x) ≤ (y++) ? (++x) : (y++));
```

| Um Outro Exemplo

```
#define SWAP(X,Y) {int temp=X; X=Y; Y=temp;}

int x = 9;
int y = 10;
SWAP(x, y);
printf("%d %d\n", x, y);
```

O que vai ser impresso?

```
#define SWAP(X,Y) {int temp=X; X=Y; Y=temp;}

int temp = 9;
int y = 10;
SWAP(temp, y);
printf("%d %d\n", temp, y);
```

E agora, o que vai ser impresso? 🤔

```
int temp = 9;
int y = 10;
{int temp = temp; temp = y; y = temp;}
printf("%d %d\n", temp, y);
```

Qual o nome disso mesmo? 🤖

- O corpo do macro é associado ao contexto do *caller* quando substituído.
- Parâmetros são avaliados no contexto onde são utilizados.

| Por Nome

Parâmetro é **avaliado toda vez que é utilizado**, mas no contexto do seu argumento (*caller*).

```
void f(by-name int a, by-name int b) {  
    b=5;  
    b=a;  
}  
int g() {  
    int i = 3;  
    f(i+1, i);  
    return i;  
}
```

Qual o valor de `i`? 🤔

| Por Necessidade

O parâmetro é **avaliado apenas na primeira vez** que é utilizado, e seu resultado é colocado em cache.

```
void f(by-need int a, by-need int b) {  
    b=a;
```

```
        b=a;
    }
    int g() {
        int i = 3;
        f(i+1, i);
        return i;
    }
```

Qual o valor de `i`? 🤔

Expansão por Macro, Por nome, e Por Necessidade:

- Avaliação é feita durante o uso do parâmetro.
 - Se o parâmetro não é utilizado, o argumento não é avaliado.
 - Exemplo: *short-circuit*
-

| Considerações Finais

1. Avaliação estrita:
 - Por valor
 - Por resultado
 - Por valor-resultado
 - Por referência
 2. Avaliação preguiçosa:
 - Por expansão de Macro
 - Por nome
 - Por necessidade
-
-

1. [Unboxing Default Argument Breaking Changes in Scikit Learn](#) ↩